Modifying LeanCoP Options On the Fly

(aka "the PAAR paper")

Introduction and Research

LeanCoP inspired a large family of provers which, with a small amount of modifications, could improve the performance of the connection prover in a large set of problems [7]. The ones that inspired this work change one property of the original LeanCoP at the initial time.

LeanCoP 2.0 [6] added several options, most notably `cut` and `scut` to restrict backtracking in connection calculus and *iterative deepening* with `comp(J)`, which iteratively limits the height 1 at a time of the tree when the prover is saturated until it reaches maximum height `J`. Afterwards it restarts iterative deepening from 1, but without any cut active.

This work also gathers ideas from other implementations of other provers from various different domains.

Letz and Stenz (2001) generally shows that reductions should be preferred over extensions in connections provers because they close the current subgoal immediately [4]. Generally shorter clauses tend to produce better solutions.

Zombori et al. (2021) introduces an entropy measurement to show how "confident" each policy should be: higher entropy suggests that the next step should be exploration, while lower entropy suggests that it should be exploitation [9]. Adding entropy regularisation improves training performance, and can make simpler models have better results than more complex ones.

Li et al. (2024) frames the reset choice in SAT solvers as an online policy problem [5]. It shows that recent search productivity should influence reset behaviour. Similar policies related to restarts are also presented in chapter 2 of Holden (2021).

Novel Idea

The three atoms in LeanCoP that can change the completeness of the current tree, `cut`, `scut`, and `comp(I)`, can be triggered dynamically using heuristics from the current tree rather than being chosen from the start.

It is possible to do a solution using reinforcement learning rather than heuristics. This would produce a connection prover that solves the same or fewer trees than existing provers such as rlCop [3] or learnCoP[citation soon!], thus making it redundant compared to the existing provers.

Ideally, an implementation that changes these atoms on the fly should be faster than both LeanCoP and RLCoP in a subset of TPTP problems.

Implementation

I use the existing connection prover implementation as a base for this code[8]. This allows me to use the existing code for hard things like Prolog parsing, and also to use PyCoP as a control group.

My code implemented 6 different strategies.

FirstAction *(control)*
: Run the first legal action (PyCoP default).

LeanCoPCut *(control)*
: Run `cut`, `scut`, and `comp(7)`, like LeanCoP.

HeuristicSelection
: Choose reductions if possible, otherwise choose the extension with the shortest clause.

EntropyCut
: Run the first legal action, but add a cut to an extension if the Shannon entropy of the amount of atoms of this is high compared to the rest, and the prover is certain that this is the solution.

AdaptiveReset
: Resets the tree if it hits 5 dead-ends in a row.

SmartCut*
: Combines the last three strategies.

Preliminary Results

A subset of 247 "representative" TPTP problems was used for testing these problems [1]. The results with any significance are fount in Table 1.

*Better name TBD.

Problem	FirstAction	LeanCopCut	HeuristicSelection	EntropyCut	AdaptiveReset	SmartCut
GEO179+3	11 987	4744	9629	11 979	2546	max-steps
GEO181+3	11 929	4778	9812	11 925	max-steps	timeout
HWV026-3	10 174	4212	max-steps	10 174	10 174	max-steps
KRS095+1	552	504	952	548	552	1014
LCL186-1	245	477	297	245	368	timeout
LCL644+1.001	8	8	2	8	8	2
LCL679+1.001	1	1	1	1	1	1
MSC015-1.005	25 401	29 037	26 257	28 251	7556	8816
NUM291+1	32 860	19 936	max-steps	32 860	32 860	max-steps
NUM300+1	11 135	7471	18 308	11 135	11 135	18 306
NUM615+1	max-steps	42 778	max-steps	max-steps	max-steps	max-steps
NUM634+3	timeout	timeout	2	timeout	timeout	2
SET062+3	24	24	26	24	24	26
SET175+3	max-steps	12 401	max-steps	max-steps	1062	1031
SET603+3	max-steps	9953	max-steps	max-steps	timeout	853
SET626+3	max-steps	35 231	max-steps	max-steps	timeout	timeout
SET886+1	max-steps	24 582	max-steps	max-steps	timeout	timeout
SET904+1	105	101	140	101	105	140
SEU121+2	max-steps	34 536	max-steps	max-steps	timeout	timeout
SWV232+1	54	54	timeout	54	54	timeout
SWV248-2	timeout	timeout	7016	6018	timeout	timeout
SWV264-2	9	9	11	9	9	11
SYN061-1	46	46	46	46	46	46
SYN582-1	15 843	11 650	40 624	15 430	timeout	timeout
SYN663-1	5651	max-steps	5588	5626	timeout	max-steps
SYN709-1	16 642	5238	11 201	16 065	max-steps	max-steps
SYN726-1	max-steps	max-steps	4468	max-steps	timeout	max-steps

Table 1: Results of running the current experiment on 247 representative TPTP problems. Only 27 of these problems finished within the time and step limit; larger experiments are coming.

The comparer runs all 6 policies in parallel, with a 30-second time limit. A larger experiment, with the entirety of TPTP and a longer time limit, will come in the future.

The results show that all three policies have marked improvements in certain problems. It's likely that with further exploration we might get even better results than the control groups.

References

[1] Karel Chvalovský and Jan Jakubův. Characteristic subsets of tptp benchmarks. In *Proceedings of the 6th Conference on Artificial Intelligence and Theorem Proving (AITP 2021)*, 2021. URL https://aitp-conference.org/2021/abstract/paper_13.pdf. Extended abstract in informal proceedings.

[2] Sean B. Holden. Machine Learning for Automated Theorem Proving: Learning to Solve SAT and QSAT. 14 (6):807–989, 2021. ISSN 1935-8237, 1935-8245. doi: 10.1561/2200000081. URL http://www.nowpublishers.com/article/Details/MAL-081.

[3] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018. URL https://papers.nips.cc/paper_files/paper/2018/hash/55acf8539596d25624059980986aaa78-Abstract.html.

[4] Reinhold Letz and Gernot Stenz. Model Elimination and Connection Tableau Procedures. In *Handbook of Automated Reasoning*, pages 2015–2114. Elsevier, 2001. ISBN 978-0-444-50813-3. doi: 10.1016/B978-044450813-3/50030-8. URL https://linkinghub.elsevier.com/retrieve/pii/B9780444508133500308.

[5] Chunxiao Li, Charlie Liu, Jonathan Chung, Zhengyang Lu, Piyush Jha, and Vijay Ganesh. A Reinforcement Learning based Reset Policy for CDCL SAT Solvers, 2024. URL http://arxiv.org/abs/2404.03753.

[6] Jens Otten. Restricting backtracking in connection calculi. 23(2–3):159–182, 2010. ISSN 09217126. doi: 10.3233/AIC-2010-0464. URL https://journals.sagepub.com/doi/full/10.3233/AIC-2010-0464.

[7] Jens Otten. 20 years of leanCoP — an overview of the provers. In *Proceedings of the 1st International*

Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023), volume 3613 of *CEUR Workshop Proceedings*, 2023. URL https://ceur-ws.org/Vol-3613/AReCCa2023_paper1.pdf.

[8] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[9] Zsolt Zombori, Josef Urban, and Miroslav Olšák. The Role of Entropy in Guiding a Connection Prover. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 218–235. Springer International Publishing, 2021. ISBN 978-3-030-86058-5 978-3-030-86059-2. doi: 10.1007/978-3-030-86059-2_13. URL https://link.springer.com/10.1007/978-3-030-86059-2_13.